

1) Блок embedding

embedding:

model_name: intfloat/multilingual-e5-large

Что делает

Определяет модель эмбедингов, которой считаются вектора для чанков и запросов.

- model_name: HuggingFace-модель sentence-transformers / e5 / bge и т.п.

Сейчас: intfloat/multilingual-e5-large — мощная мультязычная модель, даёт хорошие эмбединги, но тяжелее, чем MiniLM.

Когда менять:

- Нужна скорость → можно перейти на что-то вроде all-MiniLM-L6-v2.
- Нужна точность / мультязычность → e5 / bge большие.

Важно: при смене model_name нужно:

1. пересобрать индекс (rm -rf storage/index + python [main.py](#) build-index ...),
2. следить, чтобы chunking.tokenizer_model был согласован (или оставить как сейчас — одинаковые).

2. Блок chunking

chunking:

type: token

max_tokens: 450

overlap_tokens: 90

tokenizer_model: intfloat/multilingual-e5-large

Что делает:

Отвечает за нарезку исходных документов на чанки.

- type: token

Используется токанный чанкинг: длина измеряется в токенах конкретной модели.

- max_tokens: 450

Максимальный размер чанка в токенах.

Хочешь более длинный контекст → увеличиваешь (500–600).

Больше точность локального контекста → уменьшаешь (300–350).

- `overlap_tokens: 90`

Перекрытие между чанками (`overlap`). Сейчас ~20% от 450.

Больше `overlap` → меньше риск порвать мысль, но больше чанков, медленнее индекс и ретрив.

- `tokenizer_model: intfloat/multilingual-e5-large`

Какой токенайзер используем для измерения длины.

Логично держать его таким же, как `embedding.model_name`, чтобы «450 токенов» значили одно и то же.

Рекомендации:

- Для учебников/лонгридов формат «450 токенов, 20% `overlap`» — очень адекватный старт.
- Если `diagnostic`-скрипт показывает, что чанки получаются сильно меньше (медиана 150–200 токенов) — можно немного уменьшить `max_tokens` или `overlap`.

3. Блок `index`

`index`:

`index_dir: storage/index`

`top_k: 5`

`vector_backend: faiss_hnsw`

`faiss_hnsw_m: 32`

Что делает:

Настройки хранения векторного индекса и дефолтного `top_k`.

- `index_dir: storage/index`

Папка, куда сохраняются эмбеддинги + `faiss`/метадата.

При смене эмбеддинг-модели или логики чанкинга — индекс нужно пересобирать.

- top_k: 5

Сколько контекстов по умолчанию отдаёт pipeline, если ему явно не сказать top_k.

Для генерации ответа это «сколько чанков в финальный контекст под LLM».

- vector_backend: faiss_hnsw

Какой backend для dense-индекса:

- faiss_flat / faiss_ip — точный поиск, но медленнее;
- faiss_hnsw — approximate nearest neighbors, быстрее на больших коллекциях.
- faiss_hnsw_m: 32

Параметр плотности графа HNSW (M). Чем больше:

- лучше recall,
- выше потребление памяти и время построения.

Практически:

- Для учебника на несколько тысяч чанков faiss_flat/IndexFlatIP тоже ок, но faiss_hnsw с m=32 — хороший универсальный компромисс.
- При росте числа чанков до десятков/сотен тысяч — оставайся на HNSW и при необходимости подними m (40–48).

4. Блок retriever

retriever:

type: hybrid

hybrid_alpha: 0.5

n_candidates: 600

Что делает:

Определяет, как мы выбираем первоначальный пул кандидатов до реранков и ранкера.

- type: hybrid

Варианты:

- vector — только dense (Faiss).
- bm25 — только BM25.
- hybrid — смесь: $\text{score} = \alpha * \text{dense} + (1 - \alpha) * \text{bm25}$.

- hybrid_alpha: 0.5

Баланс между эмбедингами и BM25.

- Ближе к 1.0 → больше доверие dense.
- Ближе к 0.0 → больше доверие BM25.
- 1.5 — честная середина, как в ноуте.
- n_candidates: 600

Сколько кандидатов возвращает retriever до дальнейшей обработки.

Важно, чтобы это совпадало с логикой ноутбука:

- retriever выдаёт 600,
- 1-й перанкер забирает first_top_k = 300,
- 2-й перанкер отдаёт final_top_k = 10.

Рекомендации:

- Если у тебя 4–5к чанков (как в учебнике) — n_candidates=600 ок.
- Если база сильно больше (50–100к), можно:
- увеличить n_candidates (800–1000),
- или поиграться с hybrid_alpha (чуть сильнее опираясь на dense).

5. Блок llm

llm:

provider: local

model: "Qwen/Qwen2.5-1.5B-Instruct"

temperature: 0.2

max_tokens: 512

base_url: null

api_key_env: "" # не нужно

device: "mps" # явно говорим: M1

dtype: "float32" # если модель поддерживает; иначе можно float32

Что делает:

Настройки генератора (LLM), который отвечает на вопрос на основе контекста.

- provider: local

Используем локальную transformers-модель (через LocalTransformersClient).

- model: "Qwen/Qwen2.5-1.5B-Instruct"

Имя модели на HF. Небольшая, адекватная, тянется на M1.

- temperature: 0.2

Низкая температура → ответы более детерминированные, полезно для соревнований.

- max_tokens: 512

Максимум новых токенов в ответе.

Для коротких QA можно 256; 512 — с запасом.

- base_url, api_key_env

Игнорируются при provider: local.

- device: "mps"

Явно говорим: использовать Apple Silicon (Metal Performance Shaders).

- dtype: "float32"

На M1 это более стабильный вариант, чем float16.

Когда менять:

- Хочешь OpenAI / OpenRouter → provider: openai, заполнить model, base_url, api_key_env.
- Хочешь TGI / vLLM через HTTP → provider: tgi, задать base_url.

6. Блок reranker

reranker:

enabled: true

first_model_name: jinaai/jina-reranker-v2-base-multilingual # 1-й

first_top_k: 300 # N_FIRST_RERANK

second_model_name: BAAI/bge-reranker-v2-m3 # 2-й

final_top_k: 10

Что делает:

Двухступенчатый реранк, как в ноутбуке.

Включается, если `ranker.enabled = false`.

- `enabled: true`

Включаем pipeline реранка.

- `first_model_name`

Jina cross-encoder: мощный мультязычный реранкер.

- `first_top_k: 300`

1-й этап:

- на вход: `n_candidates = 600` от retriever,
- на выход: топ-300 по score Jina → на 2-й этап.
- `second_model_name: BAAI/bge-reranker-v2-m3`

Второй реранкер (BGE FlagReranker).

- `final_top_k: 10`

2-й этап:

- на вход: 300,
- на выход: 10 лучших чанков — именно они и идут в LLM / сабмит.

Практически:

- Эта связка (600 → 300 → 10) максимально приближает твой пайплайн к ноуту.
- Если память/время жалко:
- уменьшить `first_top_k` (например, до 200),
- или `final_top_k` (до 8), но это уже влияет на качество.

7. Блок ranker

ranker:

`enabled: false`

`model_type: lgbm`

`model_path: storage/ranker/ranker_model.pkl`

`use_cross_encoder_feature: true`

selection_mode: fixed_k

score_threshold: 0.0

dynamic_k_model_path: storage/ranker/dynamic_k_model.pkl

mmr_lambda: 0.5

mmr_top_k_candidates: 20

Что делает:

Supervised-ранкер поверх retriever'a (LightGBM / CatBoostRanker), который ты описывал для соревнований.

Сейчас:

- enabled: false → он не используется.
- Вместо него — двухступенчатый reranker.

Если включить:

- enabled: true

Тогда ranker.rank_and_select() заменяет reranker (в RAGPipeline логика: если есть ranker — используем его, а reranker игнорируем).

Остальные поля:

- model_type: lgbm

Планируемый тип ранкера (LightGBM Ranker).

- model_path

Куда/откуда грузить обученную модель.

- use_cross_encoder_feature: true

Можно скормить в фичи se_score (оценку cross-encoder'a), если ты заранее её посчитаешь.

- selection_mode: fixed_k | threshold | dynamic_k

Каким образом выбрасывать итоговый набор:

- fixed_k — всегда берём K лучших.
- threshold — берём все с score $\geq$ score_threshold.
- dynamic_k — размер набора предсказывает отдельная модель (dynamic_k_model_path).

- `mmr_lambda`, `mmr_top_k_candidates`

Настройки MMR-диверсификации (штраф за слишком похожие чанки).

Когда включать:

- Когда у тебя будет размеченный supervised датасет (`query`, `chunk`, `label`) и обученный ранкер.
- Для чисто unsupervised соревнования пока логично держать `enabled: false` и работать с two-stage reranker.

8. Блок eval

eval:

`dataset_path: data/eval/rag_eval_example.jsonl`

`results_dir: eval_results`

`eval_model: null`

Что делает:

Настройки для модуля экспериментов / оценки качества RAG.

- `dataset_path`

Путь до JSONL с примерами вида:

`{"query": "...", "answer": "...", "contexts": [...], ...}` — формат зависит от твоей реализации eval.

- `results_dir`

Куда складываются метрики, логи, JSONы с результатами.

- `eval_model`

Опциональная модель для G-Eval / LLM-based оценки (может быть gpt-4o, gpt-4.1 и т.п.).

Сейчас `null` → вероятно, используется тот же LLM или отключена автоматическая оценка.

Как всё вместе связано

Ключевые «осевые» связи:

- Чанкинг ↔ Эмбединги ↔ Индекс

- `chunking.tokenizer_model` желательно = `embedding.model_name`.
- При смене `embedding.model_name` пересобираем индекс.
- Retriever → Reranker → LLM
- Retriever: `n_candidates` = 600.
- 1-й reranker: `first_top_k` = 300.
- 2-й reranker: `final_top_k` = 10.
- `index.top_k` = 5 используется как дефолт `top_k` в pipeline, но для соревнований ты можешь передавать `--top-k` в `make_submission.py`.
- Ranker vs Reranker
- Если `ranker.enabled: true` → он главный, reranker не используется.
- Если `ranker.enabled: false` → работает двухступенчатый reranker, как в твоём ноутбуке.